Phase 4 — Descriptive Analysis: Weather Data

Descriptive and summary-statistics exhibits:

- **Tier 1 — internal QA and exploratory exhibits:** establish that the panel is complete, plausible, and contains meaningful temporal and spatial variation. Outputs are fast & minimally-styled.
- **Tier 2 — polished explanatory exhibits:** communicate the project's core weather story: long-run winter warming and county-level anomalously warm winter shocks.

PRISM is the primary weather dataset. ERA5 is a secondary robustness source and does not require a fully duplicated polished exhibit suite unless a direct PRISM–ERA5 comparison is useful.

0. Setup, definitions, and analytical conventions

Establish paths, imports, dataset configuration, labels, output directories, and common plotting settings. Document all analytical conventions before generating exhibits.

Required conventions:

- Winter is December–February (months DJF), with December assigned to the following winter year.
- With PRISM beginning in January 1981, complete DJF winters span 1982–2025.
- Construct one observation per county-winter before producing winter exhibits.
- Sum day counts and accumulated totals across winter months.
- Calculate winter temperature means using an appropriate mean, preferably weighted by observed days.
- Treat monthly variances and state variables such as snow depth according to explicitly documented aggregation rules.
- Unless otherwise specified, national and state summaries are unweighted means across counties.
- Keep freezing days (`TMIN < 32°F`) separate from extreme-cold days (`TMIN < 0°F`).

In [1]:
```
%pip install -r requirements.txt
```

```
Requirement already satisfied: earthengine-api==1.7.37 in ./.venv/lib/python
3.14/site-packages (from -r requirements.txt (line 1)) (1.7.37)
Requirement already satisfied: tqdm==4.70.0 in ./.venv/lib/python3.14/site-p
ackages (from -r requirements.txt (line 2)) (4.70.0)
Requirement already satisfied: pandas==2.3.3 in ./.venv/lib/python3.14/site-
packages (from -r requirements.txt (line 3)) (2.3.3)
Requirement already satisfied: matplotlib==3.11.1 in ./.venv/lib/python3.14/
site-packages (from -r requirements.txt (line 4)) (3.11.1)
Collecting geopandas==1.1.1 (from -r requirements.txt (line 5))
  Using cached geopandas-1.1.1-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: google-cloud-storage in ./.venv/lib/python3.1
4/site-packages (from earthengine-api==1.7.37->-r requirements.txt (line 1))
(3.13.0)
Requirement already satisfied: google-api-python-client>=2.0.2 in ./.venv/li
b/python3.14/site-packages (from earthengine-api==1.7.37->-r requirements.tx
t (line 1)) (2.198.0)
Requirement already satisfied: google-auth>=2.40.0 in ./.venv/lib/python3.1
4/site-packages (from earthengine-api==1.7.37->-r requirements.txt (line 1))
(2.56.2)
Requirement already satisfied: google-auth-httplib2>=0.3.0 in ./.venv/lib/py
thon3.14/site-packages (from earthengine-api==1.7.37->-r requirements.txt (l
ine 1)) (0.4.0)
Requirement already satisfied: httplib2<1dev,>=0.20.4 in ./.venv/lib/python
3.14/site-packages (from earthengine-api==1.7.37->-r requirements.txt (line
1)) (0.32.0)
Requirement already satisfied: requests in ./.venv/lib/python3.14/site-packa
ges (from earthengine-api==1.7.37->-r requirements.txt (line 1)) (2.34.2)
Requirement already satisfied: numpy>=1.26.0 in ./.venv/lib/python3.14/site-
packages (from pandas==2.3.3->-r requirements.txt (line 3)) (2.5.1)
Requirement already satisfied: python-dateutil>=2.8.2 in ./.venv/lib/python
3.14/site-packages (from pandas==2.3.3->-r requirements.txt (line 3)) (2.9.
0.post0)
Requirement already satisfied: pytz>=2020.1 in ./.venv/lib/python3.14/site-p
ackages (from pandas==2.3.3->-r requirements.txt (line 3)) (2026.3.post1)
Requirement already satisfied: tzdata>=2022.7 in ./.venv/lib/python3.14/site
-packages (from pandas==2.3.3->-r requirements.txt (line 3)) (2026.3)
Requirement already satisfied: contourpy>=1.0.1 in ./.venv/lib/python3.14/si
te-packages (from matplotlib==3.11.1->-r requirements.txt (line 4)) (1.3.3)
Requirement already satisfied: cycler>=0.10 in ./.venv/lib/python3.14/site-p
ackages (from matplotlib==3.11.1->-r requirements.txt (line 4)) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in ./.venv/lib/python3.14/s
ite-packages (from matplotlib==3.11.1->-r requirements.txt (line 4)) (4.63.
0)
Requirement already satisfied: kiwisolver>=1.3.1 in ./.venv/lib/python3.14/s
ite-packages (from matplotlib==3.11.1->-r requirements.txt (line 4)) (1.5.0)
Requirement already satisfied: packaging>=20.0 in ./.venv/lib/python3.14/sit
e-packages (from matplotlib==3.11.1->-r requirements.txt (line 4)) (26.3)
Requirement already satisfied: pillow>=9 in ./.venv/lib/python3.14/site-pack
ages (from matplotlib==3.11.1->-r requirements.txt (line 4)) (12.3.0)
Requirement already satisfied: pyparsing>=3 in ./.venv/lib/python3.14/site-p
ackages (from matplotlib==3.11.1->-r requirements.txt (line 4)) (3.3.2)
Requirement already satisfied: pyogrio>=0.7.2 in ./.venv/lib/python3.14/site
-packages (from geopandas==1.1.1->-r requirements.txt (line 5)) (0.13.0)
Requirement already satisfied: pyproj>=3.5.0 in ./.venv/lib/python3.14/site-
packages (from geopandas==1.1.1->-r requirements.txt (line 5)) (3.7.2)
Requirement already satisfied: shapely>=2.0.0 in ./.venv/lib/python3.14/site
```

```
-packages (from geopandas==1.1.1->-r requirements.txt (line 5)) (2.1.2)
Requirement already satisfied: google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.
0,<3.0.0,>=1.31.5 in ./.venv/lib/python3.14/site-packages (from google-api-p
ython-client>=2.0.2->earthengine-api==1.7.37->-r requirements.txt (line 1))
(2.33.0)
Requirement already satisfied: uritemplate<5,>=3.0.1 in ./.venv/lib/python3.
14/site-packages (from google-api-python-client>=2.0.2->earthengine-api==1.
7.37->-r requirements.txt (line 1)) (4.2.0)
Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.63.2 in
./.venv/lib/python3.14/site-packages (from google-api-core!=2.0.*,!=2.1.*,!=
2.2.*,!=2.3.0,<3.0.0,>=1.31.5->google-api-python-client>=2.0.2->earthengine-
api==1.7.37->-r requirements.txt (line 1)) (1.75.0)
Requirement already satisfied: protobuf<8.0.0,>=5.29.6 in ./.venv/lib/python
3.14/site-packages (from google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.
0.0,>=1.31.5->google-api-python-client>=2.0.2->earthengine-api==1.7.37->-r r
equirements.txt (line 1)) (7.35.1)
Requirement already satisfied: proto-plus<2.0.0,>=1.24.0 in ./.venv/lib/pyth
on3.14/site-packages (from google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<
3.0.0,>=1.31.5->google-api-python-client>=2.0.2->earthengine-api==1.7.37->-r
requirements.txt (line 1)) (1.28.2)
Requirement already satisfied: pyasn1-modules>=0.2.1 in ./.venv/lib/python3.
14/site-packages (from google-auth>=2.40.0->earthengine-api==1.7.37->-r requ
irements.txt (line 1)) (0.4.2)
Requirement already satisfied: cryptography>=41.0.5 in ./.venv/lib/python3.1
4/site-packages (from google-auth>=2.40.0->earthengine-api==1.7.37->-r requi
rements.txt (line 1)) (50.0.0)
Requirement already satisfied: charset_normalizer<4,>=2 in ./.venv/lib/pytho
n3.14/site-packages (from requests->earthengine-api==1.7.37->-r requirement
s.txt (line 1)) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in ./.venv/lib/python3.14/site-p
ackages (from requests->earthengine-api==1.7.37->-r requirements.txt (line
1)) (3.18)
Requirement already satisfied: urllib3<3,>=1.26 in ./.venv/lib/python3.14/si
te-packages (from requests->earthengine-api==1.7.37->-r requirements.txt (li
ne 1)) (2.7.0)
Requirement already satisfied: certifi>=2023.5.7 in ./.venv/lib/python3.14/s
ite-packages (from requests->earthengine-api==1.7.37->-r requirements.txt (l
ine 1)) (2026.7.22)
Requirement already satisfied: cffi>=2.0.0 in ./.venv/lib/python3.14/site-pa
ckages (from cryptography>=41.0.5->google-auth>=2.40.0->earthengine-api==1.
7.37->-r requirements.txt (line 1)) (2.1.1)
Requirement already satisfied: pycparser in ./.venv/lib/python3.14/site-pack
ages (from cffi>=2.0.0->cryptography>=41.0.5->google-auth>=2.40.0->earthengi
ne-api==1.7.37->-r requirements.txt (line 1)) (3.0)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in ./.venv/lib/python3.1
4/site-packages (from pyasn1-modules>=0.2.1->google-auth>=2.40.0->earthengin
e-api==1.7.37->-r requirements.txt (line 1)) (0.6.4)
Requirement already satisfied: six>=1.5 in ./.venv/lib/python3.14/site-packa
ges (from python-dateutil>=2.8.2->pandas==2.3.3->-r requirements.txt (line
3)) (1.17.0)
Requirement already satisfied: google-cloud-core<3.0.0,>=2.4.2 in ./.venv/li
b/python3.14/site-packages (from google-cloud-storage->earthengine-api==1.7.
37->-r requirements.txt (line 1)) (2.6.0)
Requirement already satisfied: google-resumable-media<3.0.0,>=2.7.2 in ./.ve
nv/lib/python3.14/site-packages (from google-cloud-storage->earthengine-api=
=1.7.37->-r requirements.txt (line 1)) (2.10.0)
```

```
Requirement already satisfied: google-crc32c<2.0.0,>=1.6.0 in ./.venv/lib/py
thon3.14/site-packages (from google-cloud-storage->earthengine-api==1.7.37->
-r requirements.txt (line 1)) (1.8.0)
Using cached geopandas-1.1.1-py3-none-any.whl (338 kB)
Installing collected packages: geopandas
  Attempting uninstall: geopandas
    Found existing installation: geopandas 1.1.4
    Uninstalling geopandas-1.1.4:
      Successfully uninstalled geopandas-1.1.4
Successfully installed geopandas-1.1.1
Note: you may need to restart the kernel to use updated packages.
```

In [2]:

```python
from dataclasses import dataclass
from pathlib import Path
import calendar
import warnings

import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages
from IPython.display import display
import numpy as np
import pandas as pd

# Make Matplotlib output render beneath each notebook cell while figures
# are also saved separately as vector PDFs.
try:
    get_ipython().run_line_magic("matplotlib", "inline")
except NameError:
    pass  # Allows the same helpers to run from a regular Python process.

# Run from codePYTHON/ on Kodama, as with the other weather scripts.
REPO_ROOT = Path.cwd().resolve().parents[0]
DATA_DIR = REPO_ROOT / "dataCSV"
TABLES_DIR = REPO_ROOT / "tables" / "weather" / "tier1"
FIGURES_DIR = REPO_ROOT / "figures" / "weather" / "tier1"
TABLES_DIR.mkdir(parents=True, exist_ok=True)
FIGURES_DIR.mkdir(parents=True, exist_ok=True)

ID_COLS = ["geoid", "state_fips", "county_fips", "county_name"]
WINTER_MONTHS = (12, 1, 2)
EXPECTED_YEARS = set(range(1981, 2026))
PRIMARY_TREND_VARS = [
    "mean_temp_c",
    "days_extremely_cold",
    "days_below_freezing_32f",
    "freeze_thaw_days",
]
SPAGHETTI_VARS = ["mean_temp_c"]
OUTLIER_Z_THRESHOLD = 5.0

STATE_FIPS_TO_NAME = {
    "01":"Alabama","02":"Alaska","04":"Arizona","05":"Arkansas","06":"Califo
    "08":"Colorado","09":"Connecticut","10":"Delaware","11":"District of Col
    "12":"Florida","13":"Georgia","15":"Hawaii","16":"Idaho","17":"Illinois"
    "18":"Indiana","19":"Iowa","20":"Kansas","21":"Kentucky","22":"Louisiana
    "23":"Maine","24":"Maryland","25":"Massachusetts","26":"Michigan",
```

```
    "27":"Minnesota","28":"Mississippi","29":"Missouri","30":"Montana",
    "31":"Nebraska","32":"Nevada","33":"New Hampshire","34":"New Jersey",
    "35":"New Mexico","36":"New York","37":"North Carolina","38":"North Dako
    "39":"Ohio","40":"Oklahoma","41":"Oregon","42":"Pennsylvania",
    "44":"Rhode Island","45":"South Carolina","46":"South Dakota",
    "47":"Tennessee","48":"Texas","49":"Utah","50":"Vermont","51":"Virginia"
    "53":"Washington","54":"West Virginia","55":"Wisconsin","56":"Wyoming",
}
VARIABLE_LABELS = {
    "mean_temp_c": "Mean winter temperature (°C)",
    "days_extremely_cold": "Extreme-cold days per winter (TMIN < 0°F)",
    "days_below_freezing_32f": "Freezing days per winter (TMIN < 32°F)",
    "freeze_thaw_days": "Freeze–thaw days per winter",
    "ppt_total": "Total precipitation (mm)",
    "heating_degree_days": "Heating degree days",
    "cooling_degree_days": "Cooling degree days",
}
VARIABLE_UNITS = {
    "mean_temp_c": "°C", "days_extremely_cold": "days",
    "days_below_freezing_32f": "days", "freeze_thaw_days": "days",
    "ppt_total": "mm", "heating_degree_days": "degree-days",
    "cooling_degree_days": "degree-days",
}

@dataclass(frozen=True)
class DatasetConfig:
    name: str
    monthly_path: Path
    derived_path: Path
    is_primary: bool = False

PRISM_CONFIG = DatasetConfig(
    "PRISM", DATA_DIR / "PRISM" / "prism_county_month.csv",
    DATA_DIR / "PRISM" / "prism_derived_weather_vars.csv", True,
)
ERA5_CONFIG = DatasetConfig(
    "ERA5", DATA_DIR / "ERA5" / "era5_county_month.csv",
    DATA_DIR / "ERA5" / "era5_derived_weather_vars.csv", False,
)
config = PRISM_CONFIG
print(f"Dataset: {config.name}")
print(f"Repository root: {REPO_ROOT}")

def variable_label(variable):
    return VARIABLE_LABELS.get(variable, variable.replace("_", " ").title())

def state_label(fips):
    code = str(fips).zfill(2)
    return f"{code} — {STATE_FIPS_TO_NAME.get(code, '(unmapped)')}"

def save_figure_pdf(fig, filename):
    path = FIGURES_DIR / filename
    fig.savefig(path, format="pdf", bbox_inches="tight")
    plt.show()
    plt.close(fig)
    print(f"Saved {path}")
```

```
    return path

def _format_table_values(table):
    formatted_table = table.copy()
    for col in formatted_table.select_dtypes(include="number"):
        formatted_table[col] = formatted_table[col].map(
            lambda x: "" if pd.isna(x) else f"{x:,.3f}" if not float(x).is_i
        )
    return formatted_table

def save_table_pdf(table, path, title, rows_per_page=32, fontsize=7):
    """Save a DataFrame as one or more vector-PDF table pages."""
    formatted_table = _format_table_values(table)
    with PdfPages(path) as pdf:
        for start in range(0, max(len(formatted_table), 1), rows_per_page):
            page = formatted_table.iloc[start:start + rows_per_page]
            fig_height = max(3.0, 0.30 * (len(page) + 4))
            fig, ax = plt.subplots(figsize=(11.7, fig_height))
            ax.axis("off")
            ax.set_title(title, loc="left", fontsize=12, pad=12)
            if page.empty:
                ax.text(
                    0.01, 0.88,
                    "No rows available. Check the preceding sample-construct
                    transform=ax.transAxes, ha="left", va="top", fontsize=10
                )
                pdf.savefig(fig, bbox_inches="tight")
                display(fig)
                plt.close(fig)
                continue
            artist = ax.table(
                cellText=page.values, colLabels=page.columns,
                loc="upper left", cellLoc="right", colLoc="center",
                bbox=[0, 0, 1, 0.94],
            )
            artist.auto_set_font_size(False)
            artist.set_fontsize(fontsize)
            artist.auto_set_column_width(col=list(range(len(page.columns))))
            pdf.savefig(fig, bbox_inches="tight")
            plt.show()
            plt.close(fig)
    print(f"Saved {path}")
    return path
```

```
Dataset: PRISM
Repository root: /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather
```

1. Load and validate the county-year-month panel

Load the raw monthly and derived-variable files, verify key uniqueness, compare overlapping QA fields, and merge on county, year, and month. Preserve clear diagnostics for duplicates, unmatched rows, missing values, and incomplete months.

In [3]:
```
def load_weather_panel(dataset_config):
    dtype = {"geoid": str, "state_fips": str, "county_fips": str}
    monthly = pd.read_csv(dataset_config.monthly_path, dtype=dtype)
    derived = pd.read_csv(dataset_config.derived_path, dtype=dtype)

    keys = ["geoid", "year", "month"]
    duplicate_monthly = int(monthly.duplicated(keys).sum())
    duplicate_derived = int(derived.duplicated(keys).sum())
    if duplicate_monthly or duplicate_derived:
        raise ValueError(
            f"Duplicate keys: monthly={duplicate_monthly:,}, derived={duplic
        )

    redundant_ids = [c for c in ID_COLS if c != "geoid"]
    qa_cols = [
        c for c in ("n_days", "expected_days", "is_incomplete", "dataset_typ
        if c in monthly.columns and c in derived.columns
    ]
    qa_mismatches = {}
    if qa_cols:
        check = monthly[keys + qa_cols].merge(
            derived[keys + qa_cols], on=keys, how="inner",
            suffixes=("_monthly", "_derived"), validate="one_to_one",
        )
        for col in qa_cols:
            left, right = check[f"{col}_monthly"], check[f"{col}_derived"]
            qa_mismatches[col] = int((left.fillna("<NA>") != right.fillna("<

    slim = derived.drop(columns=redundant_ids + qa_cols, errors="ignore")
    merged = monthly.merge(
        slim, on=keys, how="outer", indicator=True, validate="one_to_one"
    )
    merge_counts = merged["_merge"].value_counts().to_dict()
    merged = merged.drop(columns="_merge")

    diagnostics = {
        "dataset": dataset_config.name,
        "monthly_rows": len(monthly),
        "derived_rows": len(derived),
        "merged_rows": len(merged),
        "counties": merged["geoid"].nunique(),
        "first_year": int(merged["year"].min()),
        "last_year": int(merged["year"].max()),
        "duplicate_monthly_keys": duplicate_monthly,
        "duplicate_derived_keys": duplicate_derived,
        "monthly_only_rows": int(merge_counts.get("left_only", 0)),
        "derived_only_rows": int(merge_counts.get("right_only", 0)),
        "incomplete_months": int(merged.get("is_incomplete", pd.Series(False
        **{f"qa_mismatch_{key}": value for key, value in qa_mismatches.items
    }
    if diagnostics["monthly_only_rows"] or diagnostics["derived_only_rows"]:
        warnings.warn("Monthly and derived files do not match completely; ir
    return merged, diagnostics

df, load_diagnostics = load_weather_panel(config)
```

```
display(pd.DataFrame([load_diagnostics]))
display(df.head())
```

	dataset	monthly_rows	derived_rows	merged_rows	counties	first_year	last_year	dupl
0	PRISM	1678320	1678320	1678320	3108	1981	2025	

	geoid	state_fips	county_fips	county_name	year	month	n_days	ppt_total	tmean_r
0	01001	01	001	Autauga	1981	1	31	41.513720	4.92
1	01001	01	001	Autauga	1981	2	28	183.999778	9.29
2	01001	01	001	Autauga	1981	3	31	243.287245	11.80
3	01001	01	001	Autauga	1981	4	30	39.155931	19.96
4	01001	01	001	Autauga	1981	5	31	81.438487	20.52

5 rows × 27 columns

2. Construct the county-winter panel

Convert the county-month panel into one observation per county-winter using variable-specific aggregation rules. Exclude or explicitly flag partial winters so that incomplete boundary years are not presented as full winters.

In [4]:
```
NON_WEATHER_COLS = {
    *ID_COLS, "year", "month", "winter_year", "n_days", "expected_days",
    "is_incomplete", "dataset_types",
}
COUNT_OR_TOTAL_PATTERNS = (
    "days_", "heating_degree_days", "cooling_degree_days",
    "ppt_total", "total_snowfall",
)
VARIANCE_TO_MEAN = {
    "tmean_variance_c2": "tmean_mean",
    "tmin_variance_c2": "tmin_mean",
    "tmax_variance_c2": "tmax_mean",
}

def _numeric_weather_columns(frame):
    return [
        c for c in frame.columns
        if c not in NON_WEATHER_COLS and pd.api.types.is_numeric_dtype(frame
    ]

def _is_total_variable(variable):
    return variable.startswith(COUNT_OR_TOTAL_PATTERNS)

def construct_county_winter_panel(monthly):
    """Collapse county-months to complete county-winters with vectorized rul
    work = monthly.copy()
```

```
    work["winter_year"] = work["year"] + (work["month"] == 12).astype(int)
    work = work[work["month"].isin(WINTER_MONTHS)].copy()

    group_keys = ID_COLS + ["winter_year"]
    completeness = (
        work.groupby(["geoid", "winter_year"], as_index=False)
        .agg(n_months=("month", "nunique"), observed_days=("n_days", "sum"))
    )
    completeness["expected_winter_days"] = completeness["winter_year"].map(
        lambda y: 31 + calendar.monthrange(int(y), 1)[1] + 31
    )
    completeness["complete_winter"] = (
        completeness["n_months"].eq(3)
        & completeness["observed_days"].eq(completeness["expected_winter_day
    )
    if "is_incomplete" in work:
        monthly_flags = (
            work.groupby(["geoid", "winter_year"], as_index=False)["is_incom
            .any().rename(columns={"is_incomplete": "has_flagged_month"})
        )
        completeness = completeness.merge(
            monthly_flags, on=["geoid", "winter_year"], validate="one_to_one
        )
        completeness["complete_winter"] &= ~completeness["has_flagged_month"

    complete = work.merge(
        completeness[["geoid", "winter_year", "complete_winter"]],
        on=["geoid", "winter_year"], validate="many_to_one",
    )
    complete = complete[complete["complete_winter"]].copy()
    grouped = complete.groupby(group_keys, sort=True, dropna=False)
    county_winter = grouped["n_days"].sum().rename("n_days").reset_index()

    variables = _numeric_weather_columns(complete)
    variance_vars = [v for v in VARIANCE_TO_MEAN if v in variables and VARIA
    total_vars = [v for v in variables if _is_total_variable(v)]
    mean_vars = [v for v in variables if v not in total_vars and v not in va

    # Counts and accumulated quantities are additive across Dec/Jan/Feb.
    if total_vars:
        totals = grouped[total_vars].sum(min_count=1).reset_index()
        county_winter = county_winter.merge(totals, on=group_keys, validate=

    # Monthly means are combined using observed county-month day counts.
    for variable in mean_vars:
        valid = complete[variable].notna() & complete["n_days"].gt(0)
        temp = complete.loc[valid, group_keys].copy()
        temp["_weighted_value"] = complete.loc[valid, variable] * complete.l
        temp["_weight"] = complete.loc[valid, "n_days"]
        weighted = temp.groupby(group_keys, as_index=False)[["_weighted_valu
        weighted[variable] = weighted["_weighted_value"] / weighted["_weight
        county_winter = county_winter.merge(
            weighted[group_keys + [variable]], on=group_keys, how="left", va
        )

    # Combine monthly daily-sample variances exactly:
```

```
    # SS = sum[(n_m-1)s_m^2 + n_m*mean_m^2] - (sum[n_m*mean_m])^2/N.
    for variance_col in variance_vars:
        mean_col = VARIANCE_TO_MEAN[variance_col]
        valid = complete[[*group_keys, "n_days", mean_col, variance_col]].dr
        valid = valid[valid["n_days"].gt(0)].copy()
        valid["_sum_x"] = valid["n_days"] * valid[mean_col]
        valid["_sum_x2"] = (
            (valid["n_days"] - 1) * valid[variance_col]
            + valid["n_days"] * valid[mean_col].pow(2)
        )
        combined = (
            valid.groupby(group_keys, as_index=False)
            .agg(_n=("n_days", "sum"), _sum_x=("_sum_x", "sum"), _sum_x2=("_
        )
        numerator = combined["_sum_x2"] - combined["_sum_x"].pow(2) / combir
        combined[variance_col] = numerator.clip(lower=0) / (combined["_n"] -
        combined.loc[combined["_n"].le(1), variance_col] = np.nan
        county_winter = county_winter.merge(
            combined[group_keys + [variance_col]],
            on=group_keys, how="left", validate="one_to_one"
        )

    winter_diagnostics = {
        "candidate_county_winters": len(completeness),
        "complete_county_winters": int(completeness["complete_winter"].sum()
        "excluded_incomplete_county_winters": int((~completeness["complete_w
        "first_complete_winter": (
            int(county_winter["winter_year"].min()) if not county_winter.emp
        ),
        "last_complete_winter": (
            int(county_winter["winter_year"].max()) if not county_winter.emp
        ),
    }
    return county_winter, completeness, winter_diagnostics

county_winter, winter_completeness, winter_diagnostics = construct_county_wi
display(pd.DataFrame([winter_diagnostics]))
display(county_winter.head())
```

	candidate_county_winters	complete_county_winters	excluded_incomplete_county_winters
0	142968	0	142968

geoid	state_fips	county_fips	county_name	winter_year	n_days	ppt_total	days_extrem

0 rows × 24 columns

Tier 1 — Internal QA and exploratory exhibits

Tier 1 checks whether the weather panel is complete, internally coherent, and within plausible ranges. It also establishes whether the data contain meaningful variation across counties, states, and winters. These exhibits are intended primarily for internal review.

3. Data coverage and integrity table

Report the dataset, observation period, number of counties, number of county-months and county-winters, missing or incomplete records, merge mismatches, duplicate keys, and complete DJF winters retained. This table establishes the analysis sample before any substantive summaries are interpreted.

In [5]:
```
coverage = {
    **load_diagnostics,
    **winter_diagnostics,
    "expected_calendar_years": f"{min(EXPECTED_YEARS)}–{max(EXPECTED_YEARS)}
    "observed_calendar_years": f"{df['year'].min()}–{df['year'].max()}",
    "observed_complete_winters": (
        f"{county_winter['winter_year'].min()}–{county_winter['winter_year']
    ),
}
coverage_table = pd.DataFrame(
    [{"metric": key, "value": value} for key, value in coverage.items()]
)
coverage_table.to_csv(TABLES_DIR / f"{config.name.lower()}_tier1_coverage_ir
save_table_pdf(
    coverage_table,
    TABLES_DIR / f"{config.name.lower()}_tier1_coverage_integrity.pdf",
    f"{config.name}: Tier 1 data coverage and integrity",
)
display(coverage_table)
```

PRISM: Tier 1 data coverage and integrity

metric	value
dataset	PRISM
monthly_rows	1678320
derived_rows	1678320
merged_rows	1678320
counties	3108
first_year	1981
last_year	2025
duplicate_monthly_keys	0
duplicate_derived_keys	0
monthly_only_rows	0
derived_only_rows	0
incomplete_months	0
qa_mismatch_n_days	0
qa_mismatch_expected_days	0
qa_mismatch_is_incomplete	0
qa_mismatch_dataset_types	0
candidate_county_winters	142968
complete_county_winters	0
excluded_incomplete_county_winters	142968
first_complete_winter	
last_complete_winter	
expected_calendar_years	1981–2025
observed_calendar_years	1981–2025
observed_complete_winters	nan–nan

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1/prism_tier1_coverage_integrity.pdf

	metric	value
0	dataset	PRISM
1	monthly_rows	1678320
2	derived_rows	1678320
3	merged_rows	1678320
4	counties	3108
5	first_year	1981
6	last_year	2025
7	duplicate_monthly_keys	0
8	duplicate_derived_keys	0
9	monthly_only_rows	0
10	derived_only_rows	0
11	incomplete_months	0
12	qa_mismatch_n_days	0
13	qa_mismatch_expected_days	0
14	qa_mismatch_is_incomplete	0
15	qa_mismatch_dataset_types	0
16	candidate_county_winters	142968
17	complete_county_winters	0
18	excluded_incomplete_county_winters	142968
19	first_complete_winter	None
20	last_complete_winter	None
21	expected_calendar_years	1981–2025
22	observed_calendar_years	1981–2025
23	observed_complete_winters	nan–nan

4. Pooled summary-statistics table

For every raw and derived weather variable, report the unit, observation count, mean, standard deviation, minimum, p1, p5, median, p95, p99, and maximum across all county-month observations. Use the tails to identify values that require investigation.

In [6]:
```
SUMMARY_QUANTILES = [0.01, 0.05, 0.50, 0.95, 0.99]

def summarize_long(frame, group_cols=None):
    group_cols = list(group_cols or [])
    variables = _numeric_weather_columns(frame)
    records = []
```

```
    groups = [((), frame)] if not group_cols else frame.groupby(group_cols,
    for keys, group in groups:
        keys = keys if isinstance(keys, tuple) else (keys,)
        group_values = dict(zip(group_cols, keys))
        for variable in variables:
            values = group[variable].dropna()
            record = {
                **group_values,
                "variable": variable,
                "unit": VARIABLE_UNITS.get(variable, ""),
                "n": values.count(),
                "mean": values.mean(),
                "sd": values.std(),
                "min": values.min(),
                "p01": values.quantile(0.01),
                "p05": values.quantile(0.05),
                "median": values.quantile(0.50),
                "p95": values.quantile(0.95),
                "p99": values.quantile(0.99),
                "max": values.max(),
            }
            records.append(record)
    return pd.DataFrame(records)

summary_pooled = summarize_long(df)
summary_pooled.to_csv(TABLES_DIR / f"{config.name.lower()}_summary_pooled.cs
save_table_pdf(
    summary_pooled,
    TABLES_DIR / f"{config.name.lower()}_summary_pooled.pdf",
    f"{config.name}: pooled county-month summary statistics",
    rows_per_page=24,
)
display(summary_pooled)
```

PRISM: pooled county-month summary statistics

variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
ppt_total	mm	1,678,320	84.114	63.442	0	0.676	7.320	72.619	200.247	288.246	1,282.293
tmean_mean		1,678,320	12.574	9.883	-25.086	-10.581	-4.619	13.407	26.740	28.770	36.389
tmin_mean		1,678,320	6.427	9.669	-31.219	-16.077	-9.892	6.705	20.951	23.080	28.953
tmax_mean		1,678,320	18.722	10.244	-19.476	-5.270	0.345	20.077	32.727	35.357	44.151
tdmean_mean		1,678,320	6.306	9.521	-28.675	-14.023	-9.125	6.232	20.735	22.826	25.848
vpdmin_mean		1,678,320	1.078	1.174	0	0.141	0.239	0.758	2.970	6.158	25.170
vpdmax_mean		1,678,320	14.444	8.847	0.405	1.369	2.573	13.586	30.618	41.253	77.406
days_extremely_cold	days	1,678,320	0.557	2.322	0	0	0	0	4	13	31
days_below_freezing_32f	days	1,678,320	8.685	10.937	0	0	0	2	30	31	31
freeze_thaw_days	days	1,678,320	6.682	8.269	0	0	0	2	23	27	31
days_precip_above_10mm		1,678,320	2.650	2.256	0	0	0	2	7	9	26
heating_degree_days	degree-days	1,678,320	232.086	238.201	0	0	0	159.022	694.395	873.514	1,346.004
cooling_degree_days	degree-days	1,678,320	58.315	87.131	0	0	0	7.810	257.950	322.056	559.731
mean_temp_c	°C	1,678,320	12.574	9.883	-25.086	-10.581	-4.619	13.407	26.740	28.770	36.389
tmean_variance_c2		1,678,320	18.998	14.560	0.081	1.051	2.628	15.819	46.510	69.677	166.956
tmin_variance_c2		1,678,320	19.191	14.947	0.105	0.851	2.441	15.799	47.411	72.928	183.712
tmax_variance_c2		1,678,320	25.191	18.720	0.078	1.987	4.012	21.174	60.516	87.622	170.805

```
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1/prism_summary_pooled.pdf
```

	variable	unit	n	mean	sd	min	
0	ppt_total	mm	1678320	84.113505	63.442325	0.000000	0.675
1	tmean_mean		1678320	12.574239	9.883187	-25.086144	-10.580
2	tmin_mean		1678320	6.427151	9.668881	-31.218806	-16.077
3	tmax_mean		1678320	18.721636	10.243949	-19.476206	-5.269
4	tdmean_mean		1678320	6.306368	9.520643	-28.674932	-14.022
5	vpdmin_mean		1678320	1.078322	1.174098	0.000000	0.141
6	vpdmax_mean		1678320	14.443675	8.847299	0.405437	1.369
7	days_extremely_cold	days	1678320	0.557025	2.322020	0.000000	0.000
8	days_below_freezing_32f	days	1678320	8.684700	10.936999	0.000000	0.000
9	freeze_thaw_days	days	1678320	6.682357	8.269132	0.000000	0.000
10	days_precip_above_10mm		1678320	2.650242	2.256015	0.000000	0.000
11	heating_degree_days	degree-days	1678320	232.085527	238.200710	0.000000	0.000
12	cooling_degree_days	degree-days	1678320	58.315041	87.131075	0.000000	0.000
13	mean_temp_c	°C	1678320	12.574239	9.883187	-25.086144	-10.580
14	tmean_variance_c2		1678320	18.997847	14.560158	0.080759	1.051
15	tmin_variance_c2		1678320	19.190507	14.946581	0.104567	0.851
16	tmax_variance_c2		1678320	25.190853	18.719902	0.077850	1.986

5. Monthly summary-statistics table

Report the same statistics separately by calendar month. This reveals expected seasonality and helps identify month-specific values that are implausible or inconsistent with neighboring months.

In [7]:
```
summary_by_month = summarize_long(df, ["month"])
summary_by_month.to_csv(
    TABLES_DIR / f"{config.name.lower()}_summary_by_calendar_month.csv", inc
)
save_table_pdf(
    summary_by_month,
    TABLES_DIR / f"{config.name.lower()}_summary_by_calendar_month.pdf",
    f"{config.name}: summary statistics by calendar month",
    rows_per_page=30,
)
display(summary_by_month.head(24))
```

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
1	ppt_total	mm	139,860	69.302	63.659	0	0.519	4.242	54.174	181.574	292.704	908.595
1	tmean_mean		139,860	0.248	6.688	-25.086	-15.219	-10.774	0.239	10.896	14.957	22.476
1	tmin_mean		139,860	-5.282	6.471	-31.219	-21.040	-16.199	-5.103	4.872	9.062	19.001
1	tmax_mean		139,860	5.777	7.118	-19.476	-9.591	-5.517	5.627	17.273	21.349	27.192
1	tdmean_mean		139,860	-4.555	6.106	-28.675	-18.024	-14.027	-4.980	6.022	10.297	18.505
1	vpdmin_mean		139,860	0.519	0.330	0.003	0.131	0.184	0.453	1.055	1.773	5.781
1	vpdmax_mean		139,860	5.343	3.333	0.405	0.876	1.336	4.668	11.616	15.282	24.068
1	days_extremely_cold	days	139,860	2.622	4.778	0	0	0	0	13	21	31
1	days_below_freezing_32f	days	139,860	23.302	8.604	0	0	4	26	31	31	31
1	freeze_thaw_days	days	139,860	15.196	7.052	0	0	2	16	26	29	31
1	days_precip_above_10mm		139,860	2.137	2.297	0	0	0	2	6	9	26
1	heating_degree_days	degree-days	139,860	561.440	205.719	0.736	121.310	233.726	560.953	902.328	1,040.126	1,346.004
1	cooling_degree_days	degree-days	139,860	0.785	4.588	0	0	0	0	3.504	18.597	130.052
1	mean_temp_c	°C	139,860	0.248	6.688	-25.086	-15.219	-10.774	0.239	10.896	14.957	22.476
1	tmean_variance_c2		139,860	29.682	16.743	0.907	3.804	8.155	26.796	61.785	78.464	166.956
1	tmin_variance_c2		139,860	32.111	19.127	0.823	4.171	8.179	28.293	69.966	89.569	183.712
1	tmax_variance_c2		139,860	35.505	18.783	1.010	5.021	9.800	32.548	69.264	90.585	169.934
2	ppt_total	mm	139,860	69.017	62.086	0	0.778	4.886	53.464	187.165	286.554	878.315
2	tmean_mean		139,860	2.209	6.934	-21.934	-13.973	-9.589	2.373	13.096	17.142	24.089
2	tmin_mean		139,860	-3.679	6.711	-27.162	-19.987	-15.346	-3.425	6.855	11.269	20.600
2	tmax_mean		139,860	8.096	7.338	-16.756	-8.213	-3.986	8.165	19.669	23.477	29.758
2	tdmean_mean		139,860	-3.158	6.397	-24.107	-17.186	-13.371	-3.503	7.841	12.342	19.644
2	vpdmin_mean		139,860	0.594	0.387	0.006	0.148	0.207	0.514	1.193	2.103	6.549
2	vpdmax_mean		139,860	6.814	3.943	0.514	1.124	1.820	6.187	14.041	18.458	30.651
2	days_extremely_cold	days	139,860	1.890	4.021	0	0	0	0	11	18	28
2	days_below_freezing_32f	days	139,860	19.085	8.837	0	0	2	22	28	29	29
2	freeze_thaw_days	days	139,860	13.439	6.722	0	0	1	14	24	27	29
2	days_precip_above_10mm		139,860	2.118	2.178	0	0	0	2	6	9	23
2	heating_degree_days	degree-days	139,860	456.851	192.090	0	64.776	156.285	450.941	785.918	907.716	1,127.490
2	cooling_degree_days	degree-days	139,860	1.648	7.470	0	0	0	0	8.868	37.287	161.145

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
2	mean_temp_c	°C	139,860	2.209	6.934	-21.934	-13.973	-9.589	2.373	13.096	17.142	24.089
2	tmean_variance_c2		139,860	30.419	19.754	0.688	3.557	8.126	25.595	71.448	99.614	153.859
2	tmin_variance_c2		139,860	30.297	19.954	0.887	3.983	7.725	25.698	70.919	100.023	177.572
2	tmax_variance_c2		139,860	39.260	24.518	1.008	4.864	10.248	33.940	90.058	123.801	167.287
3	ppt_total	mm	139,860	82.591	59.981	0	1.985	9.945	71.936	193.633	274.971	799.133
3	tmean_mean		139,860	7.035	5.931	-12.976	-6.039	-2.658	6.975	16.611	19.825	24.967
3	tmin_mean		139,860	0.760	5.665	-18.600	-11.925	-8.364	0.622	10.202	14.053	21.070
3	tmax_mean		139,860	13.310	6.395	-7.444	-0.482	2.753	13.380	23.305	26.134	31.491
3	tdmean_mean		139,860	0.436	5.679	-17.304	-10.965	-8.103	-0.009	10.611	14.350	20.684
3	vpdmin_mean		139,860	0.849	0.520	0.044	0.200	0.296	0.748	1.672	2.808	7.885
3	vpdmax_mean		139,860	10.150	4.656	0.734	2.300	3.667	9.694	18.274	23.174	36.970
3	days_extremely_cold	days	139,860	0.417	1.502	0	0	0	0	3	8	21
3	days_below_freezing_32f	days	139,860	14.656	10.090	0	0	0	14	30	31	31
3	freeze_thaw_days	days	139,860	12.717	8.372	0	0	0	13	26	30	31
3	days_precip_above_10mm		139,860	2.610	2.189	0	0	0	2	7	9	24
3	heating_degree_days	degree-days	139,860	355.825	175.345	0	26.910	81.968	352.705	650.730	755.529	970.582
3	cooling_degree_days	degree-days	139,860	5.578	14.887	0	0	0	0	31.103	76.894	209.507
3	mean_temp_c	°C	139,860	7.035	5.931	-12.976	-6.039	-2.658	6.975	16.611	19.825	24.967
3	tmean_variance_c2		139,860	26.494	14.499	0.834	3.986	8.041	23.797	55.147	72.661	142.477
3	tmin_variance_c2		139,860	25.177	14.494	0.813	3.516	6.951	22.574	53.078	73.360	165.643
3	tmax_variance_c2		139,860	36.463	19.065	0.842	6.283	11.103	33.759	72.481	92.803	158.713
4	ppt_total	mm	139,860	87.394	58.241	0	1.812	11.889	78.221	192.685	278.938	560.502
4	tmean_mean		139,860	12.191	4.987	-5.339	1.227	4.093	12.224	20.164	22.661	27.484
4	tmin_mean		139,860	5.593	4.955	-12.523	-5.117	-2.285	5.523	13.799	17.183	22.937
4	tmax_mean		139,860	18.789	5.235	0.710	7.188	10.113	18.972	26.817	29.136	35.708
4	tdmean_mean		139,860	4.551	5.596	-17.235	-8.258	-4.466	4.466	14.003	17.015	21.656
4	vpdmin_mean		139,860	1.177	0.745	0	0.306	0.459	1.033	2.275	4.305	12.327
4	vpdmax_mean		139,860	14.461	4.979	2.201	5.841	7.530	13.982	23.280	30.223	47.305
4	days_extremely_cold	days	139,860	0.006	0.117	0	0	0	0	0	0	8
4	days_below_freezing_32f	days	139,860	6.124	7.288	0	0	0	3	22	28	30

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
4	freeze_thaw_days	days	139,860	5.911	6.930	0	0	0	3	21	27	30
4	days_precip_above_10mm		139,860	2.760	2.079	0	0	0	3	6	8	17
4	heating_degree_days	degree-days	139,860	202.579	128.461	0	3.129	21.793	189.859	427.237	513.214	710.159
4	cooling_degree_days	degree-days	139,860	18.323	29.018	0	0	0	6.135	78.906	135.776	274.532
4	mean_temp_c	°C	139,860	12.191	4.987	-5.339	1.227	4.093	12.224	20.164	22.661	27.484
4	tmean_variance_c2		139,860	21.345	10.438	0.430	4.557	7.953	19.483	41.508	53.917	84.290
4	tmin_variance_c2		139,860	19.133	8.688	0.540	3.741	6.746	18.076	35.030	44.424	69.793
4	tmax_variance_c2		139,860	31.326	16.564	0.363	4.789	9.222	28.655	63.908	80.670	115.717
5	ppt_total	mm	139,860	102.052	62.080	0	2.293	18.737	92.977	217.247	289.507	669.608
5	tmean_mean		139,860	17.445	4.386	0.443	6.908	10.108	17.534	24.265	26.065	30.811
5	tmin_mean		139,860	11.112	4.649	-6.338	-0.103	3.293	11.231	18.405	20.864	25.572
5	tmax_mean		139,860	23.778	4.351	6.771	13.398	16.546	23.853	30.462	32.570	38.206
5	tdmean_mean		139,860	10.353	5.580	-12.206	-4.909	-0.005	10.816	18.447	20.529	25.372
5	vpdmin_mean		139,860	1.306	1.075	0	0.200	0.397	1.078	2.843	6.074	15.526
5	vpdmax_mean		139,860	17.785	5.569	3.374	8.747	10.919	16.739	27.722	38.124	61.819
5	days_extremely_cold	days	139,860	0	0	0	0	0	0	0	0	0
5	days_below_freezing_32f	days	139,860	1.047	3.007	0	0	0	0	6	16	31
5	freeze_thaw_days	days	139,860	1.045	2.997	0	0	0	0	6	16	31
5	days_precip_above_10mm		139,860	3.308	2.296	0	0	0	3	7	9	17
5	heating_degree_days	degree-days	139,860	88.295	85.072	0	0	0	64.369	256.015	354.181	554.610
5	cooling_degree_days	degree-days	139,860	60.766	61.095	0	0	0	40.011	185.283	239.997	386.803
5	mean_temp_c	°C	139,860	17.445	4.386	0.443	6.908	10.108	17.534	24.265	26.065	30.811
5	tmean_variance_c2		139,860	15.103	7.894	0.272	2.502	4.565	13.642	30.113	37.948	57.145
5	tmin_variance_c2		139,860	14.588	7.112	0.322	2.717	5.106	13.369	28.200	35.918	50.790
5	tmax_variance_c2		139,860	21.040	11.959	0.088	2.735	5.064	19.211	43.140	56.552	91.305
6	ppt_total	mm	139,860	101.310	62.964	0	0.611	13.654	92.649	216.298	293.154	690.034
6	tmean_mean		139,860	22.060	3.897	5.752	11.545	14.864	22.481	27.528	29.065	32.939
6	tmin_mean		139,860	15.844	4.320	-1.341	4.041	7.692	16.340	21.905	23.558	27.096
6	tmax_mean		139,860	28.278	3.775	12.605	18.239	21.602	28.597	33.725	36.062	41.703
6	tdmean_mean		139,860	15.005	5.437	-9.843	-2.194	3.275	16.029	21.719	22.993	25.848

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
6	vpdmin_mean		139,860	1.671	1.667	0	0.144	0.333	1.255	4.355	8.989	22.353
6	vpdmax_mean		139,860	22.209	7.092	4.742	11.110	13.828	20.766	35.753	46.887	74.964
6	days_extremely_cold	days	139,860	0	0	0	0	0	0	0	0	0
6	days_below_freezing_32f	days	139,860	0.080	0.656	0	0	0	0	0	3	22
6	freeze_thaw_days	days	139,860	0.080	0.656	0	0	0	0	0	3	22
6	days_precip_above_10mm		139,860	3.316	2.384	0	0	0	3	8	10	19
6	heating_degree_days	degree-days	139,860	23.043	41.830	0	0	0	4.547	113.071	204.539	377.454
6	cooling_degree_days	degree-days	139,860	134.857	84.877	0	0	6.709	130.132	275.884	321.972	438.175
6	mean_temp_c	°C	139,860	22.060	3.897	5.752	11.545	14.864	22.481	27.528	29.065	32.939
6	tmean_variance_c2		139,860	8.979	5.470	0.147	0.956	1.808	8.130	19.156	25.110	52.151
6	tmin_variance_c2		139,860	8.655	4.824	0.200	0.966	1.929	8.133	17.637	22.225	38.458
6	tmax_variance_c2		139,860	12.687	8.368	0.240	1.556	2.892	10.793	28.982	38.935	72.852
7	ppt_total	mm	139,860	97.087	62.135	0	0.138	8.646	89.617	210.330	277.256	589.672
7	tmean_mean		139,860	24.272	3.340	8.801	15.233	18.277	24.638	28.808	30.457	36.389
7	tmin_mean		139,860	18.054	3.832	1.025	7.230	10.851	18.550	23.156	24.494	28.953
7	tmax_mean		139,860	30.492	3.239	15.332	22.035	24.972	30.753	35.290	37.639	44.151
7	tdmean_mean		139,860	17.310	4.912	-6.089	1.854	6.015	18.399	22.925	23.818	25.492
7	vpdmin_mean		139,860	1.808	1.914	0	0.115	0.292	1.178	5.611	9.380	25.170
7	vpdmax_mean		139,860	24.683	8.020	4.260	12.506	15.174	22.830	40.933	49.406	77.406
7	days_extremely_cold	days	139,860	0	0	0	0	0	0	0	0	0
7	days_below_freezing_32f	days	139,860	0.003	0.086	0	0	0	0	0	0	10
7	freeze_thaw_days	days	139,860	0.003	0.086	0	0	0	0	0	0	10
7	days_precip_above_10mm		139,860	3.150	2.402	0	0	0	3	8	10	20
7	heating_degree_days	degree-days	139,860	6.325	19.160	0	0	0	0	35.229	100.744	295.494
7	cooling_degree_days	degree-days	139,860	190.438	91.888	0	1.392	31.751	195.876	324.719	375.836	559.731
7	mean_temp_c	°C	139,860	24.272	3.340	8.801	15.233	18.277	24.638	28.808	30.457	36.389
7	tmean_variance_c2		139,860	5.375	3.377	0.111	0.574	1.032	4.827	11.778	15.244	27.825
7	tmin_variance_c2		139,860	5.240	3.512	0.105	0.461	0.791	4.726	11.715	15.241	27.563
7	tmax_variance_c2		139,860	8.011	5.102	0.078	1.071	2.044	6.918	17.988	25.492	44.965
8	ppt_total	mm	139,860	89.201	61.423	0	0.209	9.131	79.806	202.187	282.516	1,282.293

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
8	tmean_mean		139,860	23.461	3.528	8.838	15.001	17.572	23.541	28.787	30.357	35.753
8	tmin_mean		139,860	17.136	3.977	1.652	6.719	10.201	17.367	22.886	24.397	28.888
8	tmax_mean		139,860	29.786	3.453	15.168	21.873	24.210	29.805	35.399	37.640	43.597
8	tdmean_mean		139,860	16.723	4.898	-6.665	1.077	5.609	17.653	22.670	23.794	25.576
8	vpdmin_mean		139,860	1.581	1.744	0	0.091	0.230	0.980	5.049	8.441	22.071
8	vpdmax_mean		139,860	23.752	8.097	4.790	12.180	14.051	21.785	39.623	48.199	76.646
8	days_extremely_cold	days	139,860	0	0	0	0	0	0	0	0	0
8	days_below_freezing_32f	days	139,860	0.008	0.154	0	0	0	0	0	0	10
8	freeze_thaw_days	days	139,860	0.008	0.154	0	0	0	0	0	0	10
8	days_precip_above_10mm		139,860	2.799	2.184	0	0	0	3	7	9	17
8	heating_degree_days	degree-days	139,860	9.597	22.329	0	0	0	0.086	50.243	108.239	294.358
8	cooling_degree_days	degree-days	139,860	168.541	95.037	0	0.615	24.126	162.454	324.061	372.733	540.011
8	mean_temp_c	°C	139,860	23.461	3.528	8.838	15.001	17.572	23.541	28.787	30.357	35.753
8	tmean_variance_c2		139,860	6.232	3.973	0.081	0.598	1.138	5.577	13.405	18.856	36.189
8	tmin_variance_c2		139,860	6.527	4.430	0.155	0.481	0.893	5.863	14.753	19.541	33.029
8	tmax_variance_c2		139,860	9.044	5.844	0.202	1.223	2.306	7.690	20.487	28.881	61.125
9	ppt_total	mm	139,860	81.874	63.797	0	0.901	9.395	67.679	203.524	300.047	799.479
9	tmean_mean		139,860	19.591	4.019	3.910	10.167	12.972	19.606	26.040	27.704	31.617
9	tmin_mean		139,860	13.028	4.457	-2.600	2.458	5.612	12.980	20.310	22.672	26.524
9	tmax_mean		139,860	26.154	3.911	9.309	17.047	19.588	26.256	32.202	34.104	40.057
9	tdmean_mean		139,860	12.990	5.215	-8.950	-2.147	2.164	13.558	20.364	22.521	25.592
9	vpdmin_mean		139,860	1.261	1.250	0	0.113	0.243	0.882	3.593	6.261	18.675
9	vpdmax_mean		139,860	19.808	6.396	4.756	9.127	11.074	18.863	31.689	38.218	65.443
9	days_extremely_cold	days	139,860	0	0	0	0	0	0	0	0	0
9	days_below_freezing_32f	days	139,860	0.414	1.500	0	0	0	0	3	8	27
9	freeze_thaw_days	days	139,860	0.413	1.493	0	0	0	0	3	8	27
9	days_precip_above_10mm		139,860	2.466	1.980	0	0	0	2	6	8	19
9	heating_degree_days	degree-days	139,860	50.524	57.437	0	0	0	30.124	167.281	245.608	432.692
9	cooling_degree_days	degree-days	139,860	88.249	72.835	0	0	1.918	69.598	231.531	281.163	398.521
9	mean_temp_c	°C	139,860	19.591	4.019	3.910	10.167	12.972	19.606	26.040	27.704	31.617

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
9	tmean_variance_c2		139,860	13.950	8.489	0.138	1.084	3.192	12.460	30.167	41.652	72.306
9	tmin_variance_c2		139,860	15.169	8.511	0.189	0.979	3.308	13.957	30.825	41.393	59.260
9	tmax_variance_c2		139,860	18.165	12.220	0.125	1.905	4.203	15.435	41.786	61.576	101.614
10	ppt_total	mm	139,860	78.394	61.132	0	0.595	7.341	65.639	190.556	288.952	842.133
10	tmean_mean		139,860	13.347	4.641	-3.272	3.208	5.889	13.270	21.046	24.001	28.212
10	tmin_mean		139,860	6.804	4.731	-9.974	-3.272	-0.616	6.612	14.855	18.765	25.300
10	tmax_mean		139,860	19.891	4.851	3.293	8.885	11.897	19.959	27.596	29.830	35.244
10	tdmean_mean		139,860	7.041	5.337	-14.786	-6.245	-2.288	7.113	15.617	19.008	24.276
10	vpdmin_mean		139,860	0.940	0.793	0	0.156	0.265	0.737	2.242	4.113	13.797
10	vpdmax_mean		139,860	13.991	4.993	2.486	5.431	7.123	13.320	23.242	28.433	52.769
10	days_extremely_cold	days	139,860	0.008	0.127	0	0	0	0	0	0	4
10	days_below_freezing_32f	days	139,860	4.569	6.017	0	0	0	2	18	25	31
10	freeze_thaw_days	days	139,860	4.446	5.774	0	0	0	2	17	24	31
10	days_precip_above_10mm		139,860	2.411	2.009	0	0	0	2	6	8	22
10	heating_degree_days	degree-days	139,860	179.024	115.697	0	1.079	17.065	166.225	386.019	468.875	669.753
10	cooling_degree_days	degree-days	139,860	24.448	38.128	0	0	0	7.660	105.353	180.230	306.254
10	mean_temp_c	°C	139,860	13.347	4.641	-3.272	3.208	5.889	13.270	21.046	24.001	28.212
10	tmean_variance_c2		139,860	20.036	10.497	0.240	3.655	7.194	18.102	38.799	55.942	112.900
10	tmin_variance_c2		139,860	20.778	9.801	0.408	3.487	7.167	19.339	39.391	49.172	94.655
10	tmax_variance_c2		139,860	27.262	16.167	0.325	4.155	8.219	24.046	56.327	84.730	157.079
11	ppt_total	mm	139,860	74.065	63.339	0	0.419	4.933	60.402	187.236	285.426	1,039.368
11	tmean_mean		139,860	7.017	5.453	-13.099	-5.462	-1.995	7.004	15.852	19.717	26.114
11	tmin_mean		139,860	1.076	5.128	-18.316	-10.656	-7.293	1.048	9.649	14.258	23.185
11	tmax_mean		139,860	12.958	6.054	-8.205	-0.833	2.792	13.051	22.511	25.697	30.182
11	tdmean_mean		139,860	1.505	5.468	-16.915	-10.555	-7.258	1.299	10.904	15.169	22.440
11	vpdmin_mean		139,860	0.717	0.472	0.000	0.163	0.249	0.621	1.472	2.469	7.590
11	vpdmax_mean		139,860	8.751	4.020	0.646	2.026	3.027	8.349	15.840	20.048	32.952
11	days_extremely_cold	days	139,860	0.191	0.957	0	0	0	0	1	5	18
11	days_below_freezing_32f	days	139,860	13.799	9.101	0	0	0	13	29	30	30
11	freeze_thaw_days	days	139,860	12.016	7.433	0	0	0	12	24	27	30

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
11	days_precip_above_10mm		139,860	2.331	2.226	0	0	0	2	6	9	24
11	heating_degree_days	degree-days	139,860	344.019	155.923	0	29.147	97.713	340.115	609.857	713.860	942.972
11	cooling_degree_days	degree-days	139,860	4.514	14.368	0	0	0	0	25.202	74.694	233.421
11	mean_temp_c	°C	139,860	7.017	5.453	-13.099	-5.462	-1.995	7.004	15.852	19.717	26.114
11	tmean_variance_c2		139,860	23.424	10.841	0.341	4.782	8.683	22.063	42.652	56.171	141.518
11	tmin_variance_c2		139,860	23.740	11.069	0.507	4.735	8.368	22.564	43.254	57.248	137.750
11	tmax_variance_c2		139,860	30.857	15.216	0.324	5.604	11.035	28.414	58.288	78.194	164.824
12	ppt_total	mm	139,860	77.075	68.347	0	0.525	5.043	61.906	198.351	319.801	1,248.506
12	tmean_mean		139,860	2.015	6.506	-20.732	-13.760	-8.752	2.088	12.407	16.749	24.863
12	tmin_mean		139,860	-3.321	6.253	-25.627	-18.964	-13.953	-2.977	6.489	11.238	22.419
12	tmax_mean		139,860	7.351	6.974	-16.141	-8.702	-3.956	7.266	18.504	22.629	28.207
12	tdmean_mean		139,860	-2.525	6.211	-23.919	-16.521	-12.379	-2.693	8.039	12.766	22.041
12	vpdmin_mean		139,860	0.518	0.330	0.000	0.129	0.188	0.449	1.056	1.785	6.078
12	vpdmax_mean		139,860	5.577	3.243	0.408	0.912	1.450	5.024	11.536	15.158	24.458
12	days_extremely_cold	days	139,860	1.549	3.629	0	0	0	0	10	17	31
12	days_below_freezing_32f	days	139,860	21.129	9.172	0	0	3	23	31	31	31
12	freeze_thaw_days	days	139,860	14.915	6.858	0	0	2	16	25	29	31
12	days_precip_above_10mm		139,860	2.399	2.358	0	0	0	2	7	10	26
12	heating_degree_days	degree-days	139,860	507.503	198.435	0	84.125	191.575	503.617	839.649	994.889	1,211.039
12	cooling_degree_days	degree-days	139,860	1.634	7.870	0	0	0	0	8.558	37.872	202.416
12	mean_temp_c	°C	139,860	2.015	6.506	-20.732	-13.760	-8.752	2.088	12.407	16.749	24.863
12	tmean_variance_c2		139,860	26.934	15.984	0.802	4.215	8.492	23.225	57.795	80.921	141.734
12	tmin_variance_c2		139,860	28.869	16.973	0.916	4.765	8.517	25.063	61.127	84.874	162.588
12	tmax_variance_c2		139,860	32.669	18.917	1.304	5.045	10.012	28.601	67.578	98.668	170.805

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1/prism_summary_by_calendar_month.pdf

	month	variable	unit	n	mean	sd	min
0	1	ppt_total	mm	139860	69.302086	63.658517	0.000000
1	1	tmean_mean		139860	0.247689	6.688056	-25.086144
2	1	tmin_mean		139860	-5.281511	6.471120	-31.218806
3	1	tmax_mean		139860	5.776963	7.118363	-19.476206
4	1	tdmean_mean		139860	-4.555374	6.106142	-28.674932
5	1	vpdmin_mean		139860	0.518525	0.330165	0.002796
6	1	vpdmax_mean		139860	5.342757	3.333215	0.405437
7	1	days_extremely_cold	days	139860	2.621922	4.778270	0.000000
8	1	days_below_freezing_32f	days	139860	23.302174	8.604183	0.000000
9	1	freeze_thaw_days	days	139860	15.196160	7.052454	0.000000
10	1	days_precip_above_10mm		139860	2.136572	2.297472	0.000000
11	1	heating_degree_days	degree-days	139860	561.439595	205.719248	0.735994
12	1	cooling_degree_days	degree-days	139860	0.784628	4.588120	0.000000
13	1	mean_temp_c	°C	139860	0.247689	6.688056	-25.086144
14	1	tmean_variance_c2		139860	29.681628	16.742633	0.907455
15	1	tmin_variance_c2		139860	32.111103	19.126775	0.822870
16	1	tmax_variance_c2		139860	35.505357	18.782846	1.010423
17	2	ppt_total	mm	139860	69.017213	62.085681	0.000000
18	2	tmean_mean		139860	2.208621	6.933573	-21.934184
19	2	tmin_mean		139860	-3.678880	6.710893	-27.161887
20	2	tmax_mean		139860	8.096177	7.338429	-16.756080
21	2	tdmean_mean		139860	-3.158050	6.396675	-24.106918
22	2	vpdmin_mean		139860	0.593525	0.386728	0.005654
23	2	vpdmax_mean		139860	6.813523	3.943340	0.513750

6. County summary-statistics table

Calculate summary statistics separately by county. Save the complete table as a machine-readable file; show only selected examples and counties flagged by transparent outlier rules in the rendered report.

In [8]:
```
summary_by_county = summarize_long(df, ["geoid"])
county_names = df[["geoid", "state_fips", "county_name"]].drop_duplicates("g
summary_by_county = summary_by_county.merge(county_names, on="geoid", how="l
summary_by_county.to_csv(
```

```
    TABLES_DIR / f"{config.name.lower()}_summary_by_county.csv", index=False
)

# Flag county-variable means that are unusually far from the national
# distribution of county means. These are review flags, not automatic errors
county_mean_distribution = (
    summary_by_county.groupby("variable")["mean"]
    .agg(mean_across_counties="mean", std_across_counties="std")
)
flagged_counties = summary_by_county.merge(
    county_mean_distribution, left_on="variable", right_index=True,
)
flagged_counties["abs_z"] = (
    (flagged_counties["mean"] - flagged_counties["mean_across_counties"])
    / flagged_counties["std_across_counties"]
).abs()
flagged_counties = flagged_counties[
    flagged_counties["abs_z"].ge(OUTLIER_Z_THRESHOLD)
].sort_values(["variable", "abs_z"], ascending=[True, False])
flagged_counties.to_csv(
    TABLES_DIR / f"{config.name.lower()}_summary_by_county_review_flags.csv"
)
flag_columns = ["geoid", "state_fips", "county_name", "variable", "unit", "m
save_table_pdf(
    flagged_counties[flag_columns],
    TABLES_DIR / f"{config.name.lower()}_summary_by_county_review_flags.pdf"
    f"{config.name}: county-variable review flags (|z| ≥ {OUTLIER_Z_THRESHOL
    rows_per_page=28,
)
display(flagged_counties[flag_columns].head(30))
```

PRISM: county-variable review flags (|z| ≥ 5)

geoid	state_fips	county_name	variable	unit	mean	sd	min	max	abs_z
41057	41	Tillamook	ppt_total	mm	241.960	201.172	1.596	1,039.368	5.262
04027	04	Yuma	vpdmax_mean		39.906	17.896	11.310	76.646	6.039
04012	04	La Paz	vpdmax_mean		37.944	17.834	10.621	77.406	5.574
04013	04	Maricopa	vpdmax_mean		37.362	17.385	10.577	74.440	5.436
06025	06	Imperial	vpdmax_mean		37.271	16.838	10.722	74.018	5.414
32003	32	Clark	vpdmin_mean		9.028	6.030	1.099	25.170	10.519
04013	04	Maricopa	vpdmin_mean		9.017	5.623	0.966	24.439	10.503
04027	04	Yuma	vpdmin_mean		8.559	5.223	1.115	23.736	9.897
04012	04	La Paz	vpdmin_mean		8.550	5.337	0.970	22.728	9.885
06071	06	San Bernardino	vpdmin_mean		8.073	5.098	0.937	23.631	9.255
04021	04	Pinal	vpdmin_mean		7.760	4.842	0.771	20.644	8.840
06025	06	Imperial	vpdmin_mean		7.667	4.491	0.957	20.577	8.717
04015	04	Mohave	vpdmin_mean		7.414	4.793	0.959	19.823	8.383
06027	06	Inyo	vpdmin_mean		7.370	5.003	0.664	21.597	8.325
04019	04	Pima	vpdmin_mean		6.838	4.085	0.806	18.758	7.620
06065	06	Riverside	vpdmin_mean		6.733	3.801	0.812	18.937	7.481
48141	48	El Paso	vpdmin_mean		5.551	3.601	0.554	18.703	5.918
49053	49	Washington	vpdmin_mean		5.545	3.902	0.679	16.096	5.910
32017	32	Lincoln	vpdmin_mean		5.117	3.699	0.486	15.028	5.344
04025	04	Yavapai	vpdmin_mean		5.066	3.139	0.644	14.290	5.276

```
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/ti
er1/prism_summary_by_county_review_flags.pdf
```

	geoid	state_fips	county_name	variable	unit	mean	sd	
37434	41057	41	Tillamook	ppt_total	mm	241.959760	201.172258	1.596
1383	04027	04	Yuma	vpdmax_mean		39.906258	17.895697	11.310
1247	04012	04	La Paz	vpdmax_mean		37.943877	17.833950	10.621
1264	04013	04	Maricopa	vpdmax_mean		37.361513	17.385000	10.576
2879	06025	06	Imperial	vpdmax_mean		37.271014	16.838419	10.722
29143	32003	32	Clark	vpdmin_mean		9.028451	6.029890	1.099
1263	04013	04	Maricopa	vpdmin_mean		9.016967	5.622722	0.966
1382	04027	04	Yuma	vpdmin_mean		8.558738	5.223427	1.115
1246	04012	04	La Paz	vpdmin_mean		8.549660	5.336735	0.969
3269	06071	06	San Bernardino	vpdmin_mean		8.073054	5.098230	0.936
1331	04021	04	Pinal	vpdmin_mean		7.759765	4.842107	0.770
2878	06025	06	Imperial	vpdmin_mean		7.667097	4.491151	0.956
1280	04015	04	Mohave	vpdmin_mean		7.414481	4.792605	0.958
2895	06027	06	Inyo	vpdmin_mean		7.370424	5.002898	0.664
1314	04019	04	Pima	vpdmin_mean		6.837826	4.084996	0.805
3218	06065	06	Riverside	vpdmin_mean		6.732822	3.800765	0.812
43508	48141	48	El Paso	vpdmin_mean		5.551228	3.601426	0.553
47078	49053	49	Washington	vpdmin_mean		5.544981	3.902254	0.678
29262	32017	32	Lincoln	vpdmin_mean		5.117291	3.699395	0.486
1365	04025	04	Yavapai	vpdmin_mean		5.066043	3.139329	0.644

7. National winter-trend figures

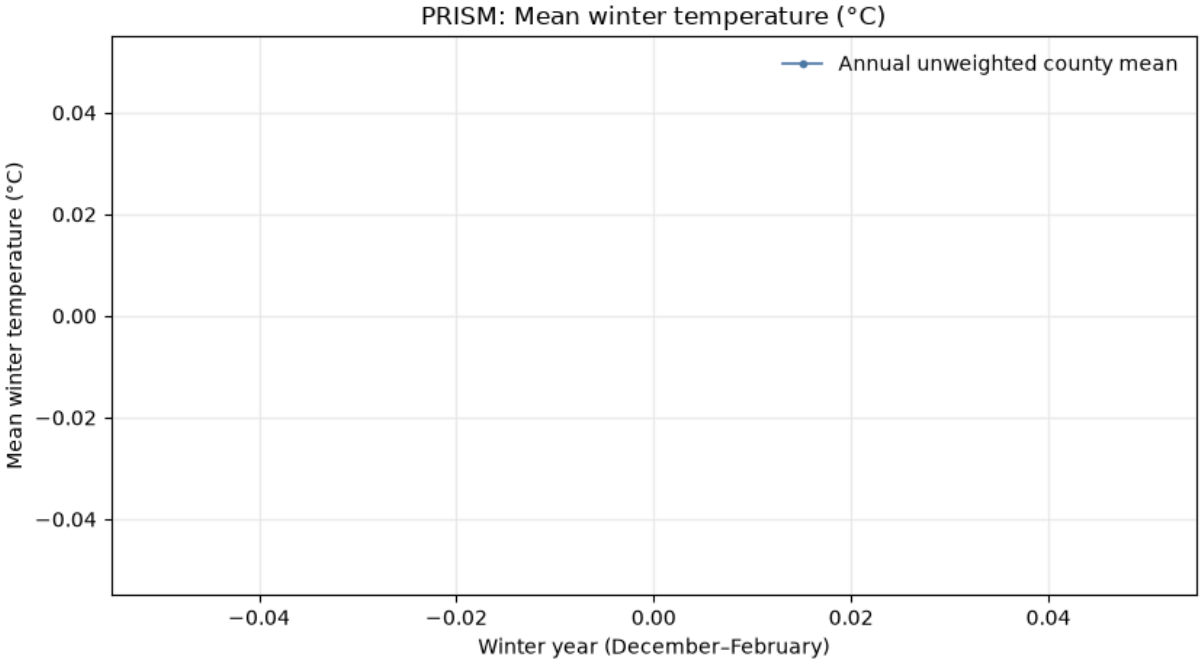
Plot the annual unweighted county mean and the full-period linear trend for each principal winter measure:

1. Mean winter temperature
2. Extreme-cold-day count (`TMIN < 0°F`)
3. Freezing-day count (`TMIN < 32°F`)
4. Freeze–thaw-day count

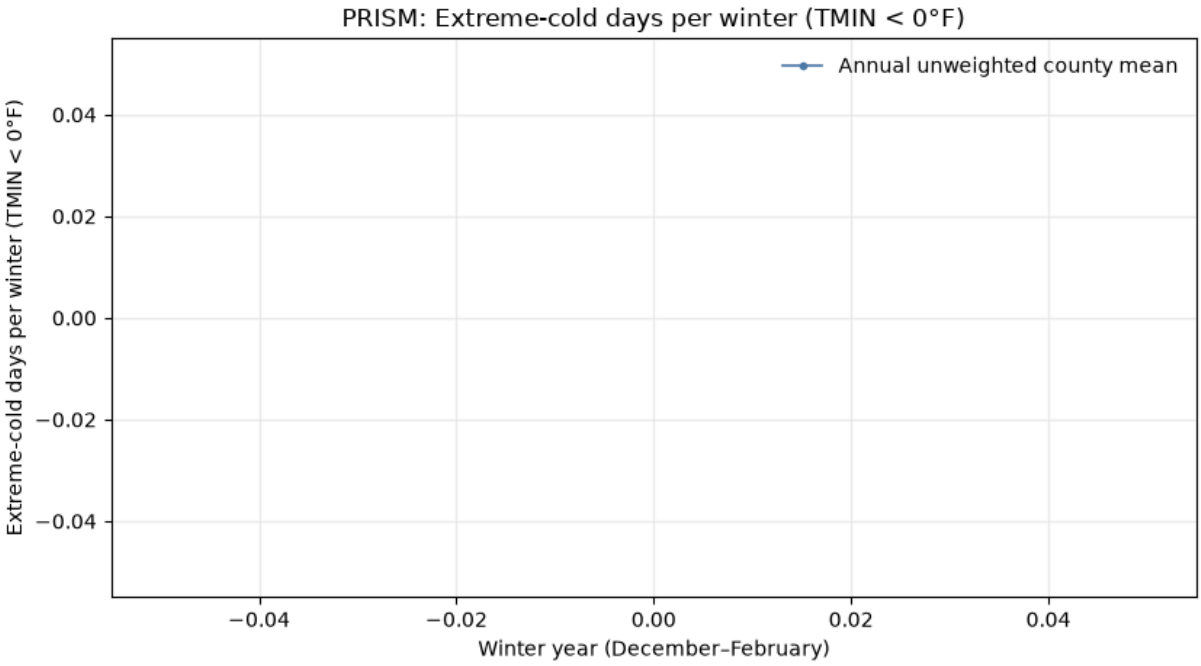
Show annual values so that common shocks remain visible; the fitted trend should supplement rather than replace them.

In [9]:
```
def _linear_trend(x, y):
    valid = pd.DataFrame({"x": x, "y": y}).dropna()
    if valid["x"].nunique() < 2:
```

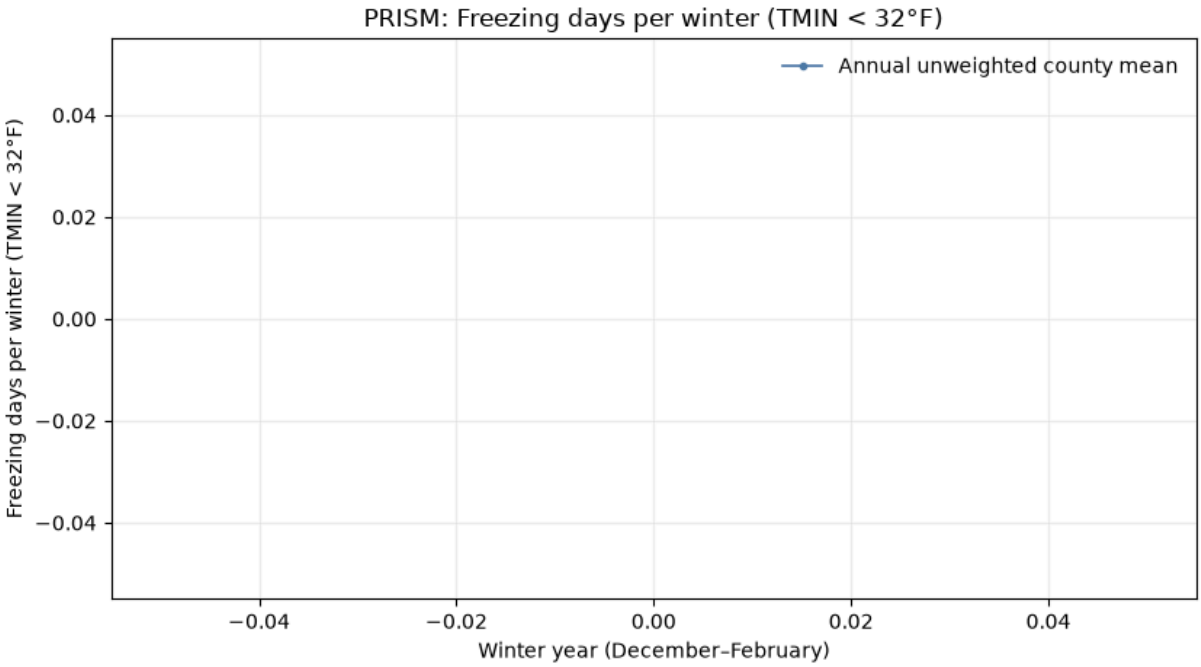
```
        return None
    slope, intercept = np.polyfit(valid["x"], valid["y"], 1)
    x_line = np.array([valid["x"].min(), valid["x"].max()])
    return x_line, intercept + slope * x_line, slope

def plot_national_winter_trend(frame, variable):
    annual = frame.groupby("winter_year", as_index=False)[variable].mean()
    trend = _linear_trend(annual["winter_year"], annual[variable])
    fig, ax = plt.subplots(figsize=(8.5, 4.8))
    ax.plot(annual["winter_year"], annual[variable], color="#4C78A8", lw=1.2
            label="Annual unweighted county mean")
    if trend is not None:
        ax.plot(trend[0], trend[1], color="black", lw=2, ls="--",
                label=f"Linear trend ({trend[2]:+.3f} per year)")
    ax.set(title=f"{config.name}: {variable_label(variable)}",
           xlabel="Winter year (December–February)", ylabel=variable_label(v
    ax.grid(alpha=0.2)
    ax.legend(frameon=False)
    fig.tight_layout()
    return save_figure_pdf(fig, f"{config.name.lower()}_national_trend_{vari

national_trend_paths = [
    plot_national_winter_trend(county_winter, variable)
    for variable in PRIMARY_TREND_VARS if variable in county_winter
]
```



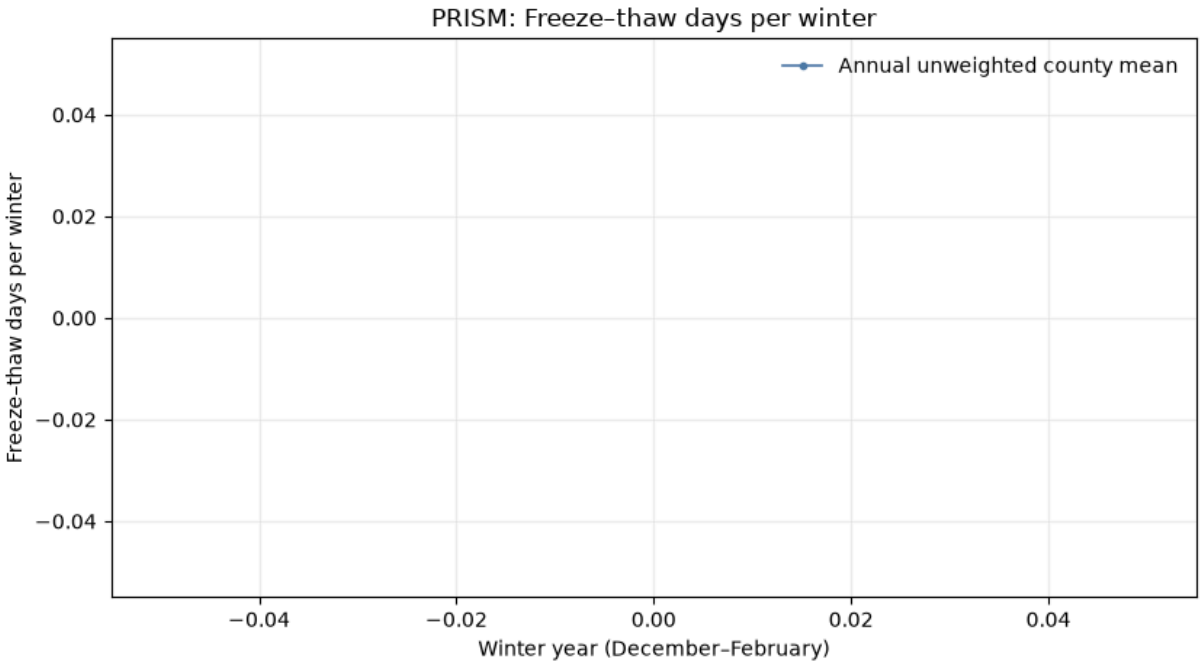

```
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/t
ier1/prism_national_trend_mean_temp_c.pdf
```



Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_national_trend_days_extremely_cold.pdf



Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_national_trend_days_below_freezing_32f.pdf




```
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/t
ier1/prism_national_trend_freeze_thaw_days.pdf
```

8. State winter-trend small multiples

Plot annual state means and full-period linear trends for the four principal winter measures, faceted by state. These figures assess geographic heterogeneity in levels, trends, and year-to-year movement. Use multi-page layouts when necessary for legibility.

In [10]:
```
def save_state_trend_multipage(frame, variable, states_per_page=12, n_cols=3
    states = sorted(frame["state_fips"].dropna().unique())
    path = FIGURES_DIR / f"{config.name.lower()}_state_trends_{variable}.pdf
    with PdfPages(path) as pdf:
        for start in range(0, len(states), states_per_page):
            page_states = states[start:start + states_per_page]
            n_rows = int(np.ceil(len(page_states) / n_cols))
            fig, axes = plt.subplots(n_rows, n_cols, figsize=(11.7, 2.65 * r
            for ax, state in zip(axes.flat, page_states):
                annual = (
                    frame[frame["state_fips"].eq(state)]
                    .groupby("winter_year", as_index=False)[variable].mean()
                )
                trend = _linear_trend(annual["winter_year"], annual[variable
                ax.plot(annual["winter_year"], annual[variable], color="#4C7
                if trend is not None:
                    ax.plot(trend[0], trend[1], color="black", lw=1.4, ls="-
                ax.set_title(state_label(state), fontsize=9)
                ax.tick_params(labelsize=7)
                ax.grid(alpha=0.15)
            for ax in list(axes.flat)[len(page_states):]:
                ax.axis("off")
            fig.suptitle(f"{config.name}: {variable_label(variable)} by stat
            fig.supxlabel("Winter year")
```

```
            fig.supylabel(variable_label(variable))
            fig.tight_layout()
            pdf.savefig(fig, bbox_inches="tight")
            plt.show()
            plt.close(fig)
    print(f"Saved {path}")
    return path

state_trend_paths = [
    save_state_trend_multipage(county_winter, variable)
    for variable in PRIMARY_TREND_VARS if variable in county_winter
]
```

```
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/t
ier1/prism_state_trends_mean_temp_c.pdf
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/t
ier1/prism_state_trends_days_extremely_cold.pdf
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/t
ier1/prism_state_trends_days_below_freezing_32f.pdf
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/t
ier1/prism_state_trends_freeze_thaw_days.pdf
```

9. County spaghetti plots by state

For each state, plot one thin, transparent line per county and a prominent annual state-mean line. Optionally add a distinct dashed linear trend. Use these figures to inspect within-county variation, between-county dispersion, common shocks, outliers, discontinuities, and changes in dispersion over time.

In [11]:
```
def save_spaghetti_multipage(frame, variable, states_per_page=6, n_cols=2):
    states = sorted(frame["state_fips"].dropna().unique())
    path = FIGURES_DIR / f"{config.name.lower()}_county_spaghetti_{variable}
    with PdfPages(path) as pdf:
        for start in range(0, len(states), states_per_page):
            page_states = states[start:start + states_per_page]
            n_rows = int(np.ceil(len(page_states) / n_cols))
            fig, axes = plt.subplots(n_rows, n_cols, figsize=(11.7, 3.3 * n_
            for ax, state in zip(axes.flat, page_states):
                state_frame = frame[frame["state_fips"].eq(state)]
                for _, county in state_frame.groupby("geoid"):
                    county = county.sort_values("winter_year")
                    ax.plot(county["winter_year"], county[variable],
                            color="#4C78A8", alpha=0.18, lw=0.45)
                annual_mean = state_frame.groupby("winter_year", as_index=Fa
                ax.plot(annual_mean["winter_year"], annual_mean[variable],
                        color="black", lw=1.8, label="Annual state mean")
                trend = _linear_trend(annual_mean["winter_year"], annual_mea
                if trend is not None:
                    ax.plot(trend[0], trend[1], color="#E45756", lw=1.3, ls=
                            label="State linear trend")
                ax.set_title(f"{state_label(state)} (n={state_frame['geoid']
                ax.tick_params(labelsize=7)
                ax.grid(alpha=0.12)
            for ax in list(axes.flat)[len(page_states):]:
```

```
                ax.axis("off")
            handles, labels = axes.flat[0].get_legend_handles_labels()
            if handles:
                fig.legend(handles, labels, loc="upper center", ncol=2, fram
            fig.suptitle(
                f"{config.name}: county trajectories — {variable_label(varia
                fontsize=13, y=0.995,
            )
            fig.supxlabel("Winter year")
            fig.supylabel(variable_label(variable))
            fig.tight_layout(rect=[0, 0, 1, 0.97])
            pdf.savefig(fig, bbox_inches="tight")
            plt.show()
            plt.close(fig)
    print(f"Saved {path}")
    return path

spaghetti_paths = [
    save_spaghetti_multipage(county_winter, variable)
    for variable in SPAGHETTI_VARS if variable in county_winter
]
```

```
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/t
ier1/prism_county_spaghetti_mean_temp_c.pdf
```

10. Decade-distribution plots by state

Compare county-level distributions of mean winter temperature across decades within each state. Use these exploratory plots to determine whether the distribution shifts in location, spread, or shape—not merely whether its mean changes.

In [12]:
```
def decade_label(year):
    start = (int(year) // 10) * 10
    return f"{start}s"

county_winter["decade"] = county_winter["winter_year"].map(decade_label)

def save_decade_distributions(frame, variable="mean_temp_c", states_per_page
    states = sorted(frame["state_fips"].dropna().unique())
    decades = sorted(frame["decade"].unique())
    path = FIGURES_DIR / f"{config.name.lower()}_state_decade_distributions_
    with PdfPages(path) as pdf:
        for start in range(0, len(states), states_per_page):
            page_states = states[start:start + states_per_page]
            n_rows = int(np.ceil(len(page_states) / n_cols))
            fig, axes = plt.subplots(n_rows, n_cols, figsize=(11.7, 2.75 * r
            for ax, state in zip(axes.flat, page_states):
                sub = frame[frame["state_fips"].eq(state)]
                values = [sub.loc[sub["decade"].eq(decade), variable].dropna
                ax.boxplot(values, labels=decades, showfliers=False, patch_a
                           boxprops={"facecolor":"#9ECAE1", "edgecolor":"#4C
                           medianprops={"color":"black"})
                ax.set_title(state_label(state), fontsize=9)
                ax.tick_params(axis="x", rotation=45, labelsize=7)
```

```
                ax.tick_params(axis="y", labelsize=7)
                ax.grid(axis="y", alpha=0.15)
            for ax in list(axes.flat)[len(page_states):]:
                ax.axis("off")
            fig.suptitle(f"{config.name}: decade distributions of {variable_
            fig.supylabel(variable_label(variable))
            fig.tight_layout()
            pdf.savefig(fig, bbox_inches="tight")
            plt.show()
            plt.close(fig)
    print(f"Saved {path}")
    return path

decade_distribution_path = save_decade_distributions(county_winter)
```

```
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/t
ier1/prism_state_decade_distributions_mean_temp_c.pdf
```

11. Interannual-variability summary

Calculate the standard deviation of county-level mean winter temperature across winters and summarize its distribution nationally and by state. This measure captures general year-to-year variability; it does not specifically identify anomalously warm winters.

In [13]:
```
if county_winter.empty:
    warnings.warn(
        "The county-winter panel is empty; interannual-variability outputs w
        "Review the completeness diagnostics from Section 2 before interpret
    )

county_variability = (
    county_winter.groupby(["geoid", "state_fips", "county_name"], as_index=F
    .agg(
        winter_temp_sd=("mean_temp_c", "std"),
        winter_temp_mean=("mean_temp_c", "mean"),
        n_winters=("mean_temp_c", "count"),
    )
)
county_variability["winter_temp_cv"] = (
    county_variability["winter_temp_sd"] / county_variability["winter_temp_m
)
county_variability.to_csv(
    TABLES_DIR / f"{config.name.lower()}_county_interannual_variability.csv"
)

variability_national = summarize_long(
    county_variability.rename(columns={"winter_temp_sd": "interannual_sd_c"}
)
variability_by_state = (
    county_variability.groupby("state_fips")["winter_temp_sd"]
    .agg(n_counties="count", mean="mean", sd="std", min="min", median="media
    .reset_index()
)
variability_by_state["state"] = variability_by_state["state_fips"].map(state
```

```
variability_by_state = variability_by_state[
    ["state_fips", "state", "n_counties", "mean", "sd", "min", "median", "ma
]
variability_by_state.to_csv(
    TABLES_DIR / f"{config.name.lower()}_interannual_variability_by_state.cs
)
save_table_pdf(
    variability_by_state,
    TABLES_DIR / f"{config.name.lower()}_interannual_variability_by_state.pd
    f"{config.name}: interannual SD of mean winter temperature by state",
    rows_per_page=28,
)
display(variability_by_state)
```

```
/tmp/ipykernel_782770/1079007676.py:2: UserWarning: The county-winter panel
is empty; interannual-variability outputs will have no rows. Review the comp
leteness diagnostics from Section 2 before interpreting this exhibit.
  warnings.warn(
```

PRISM: interannual SD of mean winter temperature by state

No rows available. Check the preceding sample-construction diagnostics.

```
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/ti
er1/prism_interannual_variability_by_state.pdf
```

state_fips	state	n_counties	mean	sd	min	median	max

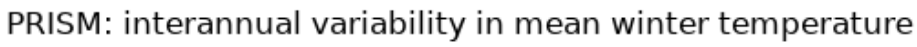
12. Interannual-variability choropleth

Map the standard deviation of mean winter temperature across winters by county. The purpose is to identify regions with high general interannual variability and distinguish that concept from long-run warming and anomalous-warm-winter frequency.

In [14]:
```
COUNTY_GEOMETRY_CANDIDATES = [
    REPO_ROOT / "dataRAW" / "shapefiles" / "cb_2023_us_county_20m.shp",
    REPO_ROOT / "dataRAW" / "cb_2023_us_county_20m.shp",
]
CENSUS_COUNTY_URL = "https://www2.census.gov/geo/tiger/GENZ2023/shp/cb_2023_

def load_county_geometry():
    try:
        import geopandas as gpd
    except ImportError as exc:
        raise ImportError(
            "Tier 1 choropleths require GeoPandas. Install the project deper
            "in this notebook kernel with: %pip install -r ../requirements.t
```

```
            "Then restart the kernel and rerun the notebook."
        ) from exc
    local = next((path for path in COUNTY_GEOMETRY_CANDIDATES if path.exists
    source = local if local is not None else CENSUS_COUNTY_URL
    geometry = gpd.read_file(source)
    geoid_col = next((c for c in ("GEOID", "GEOID20", "geoid") if c in geome
    if geoid_col is None:
        raise KeyError("County geometry does not contain a recognized GEOID
    geometry["geoid"] = geometry[geoid_col].astype(str).str.zfill(5)
    return geometry[["geoid", "geometry"]]

county_geometry = load_county_geometry()
variability_map = county_geometry.merge(
    county_variability[["geoid", "winter_temp_sd"]], on="geoid", how="inner"
)
variability_map = variability_map.to_crs("EPSG:5070")
fig, ax = plt.subplots(figsize=(11, 7))
variability_map.plot(
    column="winter_temp_sd", cmap="viridis", linewidth=0,
    legend=True, legend_kwds={"label": "SD of mean winter temperature (°C)"},
    missing_kwds={"color": "lightgrey"}, ax=ax,
)
ax.set_title(f"{config.name}: interannual variability in mean winter tempera
ax.axis("off")
fig.tight_layout()
interannual_variability_map_path = save_figure_pdf(
    fig, f"{config.name.lower()}_interannual_variability_choropleth.pdf"
)
```

```
/tmp/ipykernel_782770/2589441219.py:31: UserWarning: The GeoDataFrame you ar
e attempting to plot is empty. Nothing has been displayed.
  variability_map.plot(
```




```
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/t
ier1/prism_interannual_variability_choropleth.pdf
```

13. Tier 1 QA findings and flagged observations

Summarize the main QA findings in plain language. List implausible values, unusual counties or winters, merge or coverage issues, and decisions requiring follow-up. Distinguish confirmed data problems from genuine but unusual weather observations.

In [15]:
```
qa_findings = [
    {
        "check": "Duplicate county-year-month keys",
        "result": load_diagnostics["duplicate_monthly_keys"] + load_diagnost
        "status": "PASS" if not (load_diagnostics["duplicate_monthly_keys"]
    },
    {
```

```
        "check": "Rows appearing in only one merge source",
        "result": load_diagnostics["monthly_only_rows"] + load_diagnostics["
        "status": "PASS" if not (load_diagnostics["monthly_only_rows"] + loa
    },
    {
        "check": "Incomplete county-months",
        "result": load_diagnostics["incomplete_months"],
        "status": "PASS" if not load_diagnostics["incomplete_months"] else "
    },
    {
        "check": "Excluded incomplete county-winters",
        "result": winter_diagnostics["excluded_incomplete_county_winters"],
        "status": "INFO",
    },
    {
        "check": f"County-variable mean flags (|z| ≥ {OUTLIER_Z_THRESHOLD:g}
        "result": len(flagged_counties),
        "status": "REVIEW" if len(flagged_counties) else "PASS",
    },
]
qa_findings = pd.DataFrame(qa_findings)
qa_findings.to_csv(TABLES_DIR / f"{config.name.lower()}_tier1_qa_findings.cs
save_table_pdf(
    qa_findings,
    TABLES_DIR / f"{config.name.lower()}_tier1_qa_findings.pdf",
    f"{config.name}: Tier 1 QA findings",
)
display(qa_findings)

print("Tier 1 outputs complete.")
print(f"Tables: {TABLES_DIR}")
print(f"Figures: {FIGURES_DIR}")
```

PRISM: Tier 1 QA findings

check	result	status
Duplicate county-year-month keys	0	PASS
Rows appearing in only one merge source	0	PASS
Incomplete county-months	0	PASS
Excluded incomplete county-winters	142,968	INFO
County-variable mean flags (|z| ≥ 5)	20	REVIEW

```
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/ti
er1/prism_tier1_qa_findings.pdf
```

	check	result	status
0	Duplicate county-year-month keys	0	PASS
1	Rows appearing in only one merge source	0	PASS
2	Incomplete county-months	0	PASS
3	Excluded incomplete county-winters	142968	INFO
4	County-variable mean flags (|z| ≥ 5)	20	REVIEW

```
Tier 1 outputs complete.
Tables: /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/
tier1
Figures: /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weathe
r/tier1
```

Tier 2 — Polished explanatory exhibits

Tier 2 communicates the two weather forces central to the project's causal story: long-run winter warming and short-run anomalously warm winter shocks. These exhibits should use publication-quality labels, units, annotations, legends, colorblind-accessible palettes, and vector output where possible.

14. National long-run warming exhibit

Present a polished national figure showing annual mean winter temperature and its long-run trend over the complete-winter sample. Clearly identify the geographic weighting convention and retain annual observations so anomalous winters remain visible.

In [ ]:

15. State long-run warming exhibit

Present polished state-level small multiples showing geographic heterogeneity in winter-temperature trends. Consider displaying trend slopes or early-to-recent changes if annual series make the panels too dense.

In [ ]:

16. Early-versus-recent winter-temperature choropleths

Map county mean winter temperature for a clearly defined early period and recent period. Use identical classification rules and color limits so that the two maps are directly comparable. Document the selected periods.

In [ ]:

17. Change-in-winter-temperature choropleth

Map the county-level change in mean winter temperature between the early and recent periods. This exhibit should directly identify which places experienced the greatest long-

run winter warming.

In [ ]:

18. Extreme-cold-day choropleth

Map the mean annual number of winter days with minimum temperature below 0°F, or the change in that count between the early and recent periods. Label this measure **extreme-cold-day count**, not Winter Severity Index. A complete Wisconsin-style WSI would require additional components such as deep-snow days.

In [ ]:

19. Define anomalously warm winters

Specify and justify the provisional anomaly definition before producing anomaly exhibits. Document the winter metric, county-specific reference distribution, treatment of the long-run trend, threshold, minimum coverage, and whether results are expressed as counts or shares. Candidate definitions require Eyal's confirmation before being treated as final.

In [ ]:

20. Anomalous-warm-winter frequency choropleth

Map the number or share of winters classified as anomalously warm within each county. This exhibit captures the geographic concentration of short-run warm shocks rather than general variability or long-run warming. Clearly label the anomaly definition as provisional or confirmed.

In [ ]:

21. Anomalous-winter time-series exhibit

Show the national or state-level frequency of anomalously warm county-winters over time. Use this figure to distinguish unusual short-run shocks from the gradual long-run warming trend.

In [ ]:

22. Polished decade-distribution exhibit

Select the most informative national, regional, or state examples from the Tier 1 decade-distribution plots. Use them to demonstrate whether climate change shifts the location, spread, or shape of the winter-temperature distribution.

In [ ]:

23. Optional targeted PRISM–ERA5 comparison

If needed, compare selected PRISM and ERA5 measures to assess robustness and agreement. Prioritize targeted comparisons over duplicating the complete PRISM exhibit suite.

In [ ]:

24. Exhibit index, output manifest, and decisions for Eyal

End the report with an ordered exhibit index and a manifest of saved tables and figures. Summarize unresolved methodological choices—especially anomaly definition, weighting, comparison periods, and any questionable observations—for efficient PI review.

In [ ]: